ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

HONORIS UNITED UNIVERSITIES

RAPPORT DE PROJET DE FIN D'ANNÉE

FOODAID

PLATEFORME INTELLIGENTE DE GESTION DES DONS ALIMENTAIRES

RÉALISÉ PAR :

Kawtar Chemaou

Rania Lamsakhar

FILIÈRE :

3IIR11

ENCADRÉ PAR :

Mme Benyoussef Marwa

ANNÉE UNIVERSITAIRE :

2025 – 2026

Table des matières

Liste des abréviations **vi**

Dédicaces **vii**

Remerciements **viii**

Résumé **ix**

Abstract **x**

1 Contexte général du projet **1**

Introduction . . . 1

1.1 Présentation générale du projet . . . 1

1.1.1 Contexte du projet . . . 1

1.1.2 Problématique . . . 1

1.1.3 Objectifs du projet . . . 1

1.1.4 Solution proposée . . . 2

1.2 Étude de l'existant . . . 2

1.2.1 Analyse des solutions existantes . . . 2

1.2.2 Limites des solutions actuelles . . . 2

1.2.3 Valeur ajoutée de FoodAid . . . 2

2 Analyse et conception de la plateforme FoodAid **3**

Introduction . . . 3

2.1 Analyse des besoins . . . 3

2.1.1 Périmètre du projet . . . 3

2.1.2 Acteurs de la plateforme . . . 3

2.1.3 Diagrammes des cas d'utilisation . . . 3

2.1.4 Diagrammes de séquence . . . 9

2.1.5 Diagramme de classes . . . 21

2.2 Conception de la base de données . . . 22

2.2.1 Modèle conceptuel . . . 22

2.2.2 Liste des tables de la base de données . . . 23

2.2.3 Tables principales . . . 24

3 Réalisation et interfaces de l'application **25**

Introduction . . . 25

3.1 Environnement de développement 25
3.1.1 Environnement matériel 25
3.1.2 Environnement logiciel 25
3.2 Description des besoins techniques 26
3.2.1 Architecture générale de l'application 26
3.2.2 Technologies utilisées 26
3.2.3 Gestion des erreurs d'authentification et alternatives 27
3.2.4 Gestion de la perte de messages et alternatives réseau 28
3.2.5 Intégration du chatbot avec système RAG 29
3.2.6 WebSocket pour le suivi en temps réel 30
3.2.7 Outils de développement 30
3.2.8 Structure du projet 30
3.3 Implémentation de la plateforme FoodAid 31
3.3.1 Mise en place du back-end 31
3.3.2 Mise en place du front-end 32
3.3.3 Mise en place de l'authentification 32
3.3.4 Intégration du QR Code 33
3.3.5 Intégration du suivi en temps réel 33
3.4 Présentation des interfaces de l'application 33
3.5 Tests fonctionnels 36
3.5.1 Ordre des tests 36
3.5.2 Tests de gestion des dons 36
3.5.3 Tests de livraison et validation QR Code 37
Conclusion du chapitre 38

Conclusion générale et perspectives **39**

Références bibliographiques **40**

Table des figures

2.1 Diagramme des cas d'utilisation du livreur 4
2.2 Diagramme des cas d'utilisation du donateur 5
2.3 Diagramme des cas d'utilisation de l'administrateur 6
2.4 Diagramme des cas d'utilisation du volontaire 7
2.5 Diagramme des cas d'utilisation de l'association 8
2.6 Diagramme de séquence - Authentification de l'administrateur avec gestion des erreurs 9
2.7 Diagramme de séquence - Activation/désactivation/gestion et affichage de tous types des utilisateurs 10
2.8 Diagramme de séquence - Gestion des missions par l'administrateur 10
2.9 Diagramme de séquence - Vérification des dons nécessitant des volontaires 11
2.10 Diagramme de séquence - Gestion des missions par le livreur 12
2.11 Diagramme de séquence - Communication avec la base de données et système RAG 13
2.12 Diagramme de séquence - Consulter des missions affectées avec un code QR de positionnement 13
2.13 Diagramme de séquence - Authentification du donateur avec gestion des erreurs 15
2.14 Diagramme de séquence - Consultation de la vue IA des besoins globaux . 16
2.15 Diagramme de séquence - Communication avec le support 16
2.16 Diagramme de séquence - Historique des dons et le suivi en temps réel de livreur concerné 17
2.17 Diagramme de séquence - Authentification de l'association avec gestion des erreurs 18
2.18 Diagramme de séquence - Gestion des bénéficiaires 19
2.19 Diagramme de séquence - Déclaration de demande de dons 19
2.20 Diagramme de séquence - Suivi en temps réel de livreur et demande historique 20
2.21 Diagramme de séquence - Urgence et suivi des dons 20
2.22 Diagramme de classes de la plateforme FoodAid (partie 1) 21
2.23 Diagramme de classes de la plateforme FoodAid (partie 2) 21
2.24 Diagramme de classes de la plateforme FoodAid (partie 3) 22

3.1 Page de connexion de la plateforme FoodAid 33
3.2 Page d'inscription avec choix du rôle 34
3.3 Tableau de bord administrateur - Vue d'ensemble 34
3.4 Tableau de bord donateur - Gestion des dons 34

3.5 Tableau de bord association - Suivi des aides 35
3.6 Tableau de bord livreur - Missions de livraison 35
3.7 Tableau de bord volontaire - Actions terrain 35

Liste des tableaux

2.1 Synthèse des cas d'utilisation par acteur 9
2.2 Principales relations de la base de données 22
2.3 Liste complète des tables de la base de données FoodAid 23

3.1 Technologies utilisées par couche 26

Liste des abréviations

Abréviation	Signification
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
GPS	Global Positioning System
HTTP	HyperText Transfer Protocol
IA	Intelligence Artificielle
JPA	Java Persistence API
JWT	JSON Web Token
PERT	Program Evaluation and Review Technique
QR Code	Quick Response Code
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language
WebSocket	Protocole de communication bidirectionnelle temps réel

Dédicaces

À nos familles, pour leur soutien et leur patience.
À tous ceux qui luttent contre le gaspillage alimentaire et œuvrent pour la solidarité.
À toutes les personnes qui ont contribué à ce projet.

L'équipe FoodAid

Remerciements

Nous remercions chaleureusement notre encadrante, **Madame BENYOUSSEF Marwa**, pour son accompagnement, ses précieux conseils et sa disponibilité tout au long de ce projet.

Nous remercions également l'équipe pédagogique de l'EMSI ainsi que toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce travail.

L'équipe FoodAid

Résumé

FoodAid est une plateforme web intelligente de gestion et distribution des dons alimentaires. Elle met en relation donateurs, associations, livreurs, volontaires, bénéficiaires et administrateurs pour lutter contre le gaspillage alimentaire.

La plateforme intègre la géolocalisation, les notifications, la messagerie instantanée, la validation par QR Code et des services d'intelligence artificielle (assistant, recettes, rapports d'aide à la décision).

Ce rapport présente l'analyse, la conception, l'implémentation et les tests de la solution.

Mots-clés : dons alimentaires, gaspillage, solidarité, IA, Spring Boot, React.

Abstract

FoodAid is an intelligent web platform for managing food donations, connecting donors, charities, delivery workers, volunteers, beneficiaries and administrators to reduce food waste.

Built with React, Spring Boot and PostgreSQL, the platform includes user management, donation tracking, delivery missions, GPS, QR Code validation, chat, notifications and AI services (assistant, recipes, image generation, logistics reports).

This report covers the analysis, design, implementation and testing of the solution.

Keywords : food donations, food waste, charity, Spring Boot, React, AI, QR Code.

1 Contexte général du projet

Introduction

Face à l'ampleur du gaspillage alimentaire et aux difficultés logistiques rencontrées par les associations, le projet FoodAid propose une solution web centralisée pour gérer les dons alimentaires, leur transport et leur distribution.

Ce chapitre présente le contexte, la problématique, les objectifs et la solution proposée.

1.1 Présentation générale du projet

1.1.1 Contexte du projet

FoodAid est une plateforme web solidaire facilitant la gestion et la distribution des dons alimentaires en mettant en relation donateurs, associations, livreurs, volontaires, bénéficiaires et administrateurs.

1.1.2 Problématique

Comment concevoir une plateforme centralisée, sécurisée et optimisée pour la gestion des dons alimentaires, assurant le suivi des livraisons et la coordination entre acteurs ?

Les difficultés identifiées sont :

— Absence d'un espace unique regroupant tous les acteurs ;

— Manque de traçabilité des dons et des livraisons ;

— Difficulté de coordination et de communication entre intervenants.

1.1.3 Objectifs du projet

FoodAid vise à :

— Centraliser et sécuriser la gestion des utilisateurs et des dons ;

— Assurer le suivi des livraisons (GPS, QR Code, notifications) ;

— Faciliter la communication entre acteurs ;

— Intégrer des services d'intelligence artificielle (assistant, recettes, rapports).

1.1.4 Solution proposée

FoodAid est une application web composée d'un frontend React.js et d'un backend Spring Boot exposant des API REST sécurisées, connecté à une base PostgreSQL.

1.2 Étude de l'existant

1.2.1 Analyse des solutions existantes

Des solutions comme *Too Good To Go* (anti-gaspillage) ou *Restos du Cœur* (distribution) couvrent des aspects spécifiques, mais aucune ne propose une gestion intégrée de toute la chaîne du don alimentaire. Les associations utilisent encore souvent des outils disparates (tableurs, appels, formulaires), limitant la traçabilité.

1.2.2 Limites des solutions actuelles

— Absence d'intégration entre création, validation, livraison et confirmation du don ;

— Pas de suivi GPS ni de QR Code pour valider les livraisons ;

— Pas d'assistant intelligent pour aider à la décision.

1.2.3 Valeur ajoutée de FoodAid

— Centralisation de tous les acteurs sur une même plateforme ;

— Traçabilité complète des dons et livraisons ;

— Géolocalisation et validation par QR Code ;

— Services IA pour l'assistance et l'aide à la décision.

2 Analyse et conception de la plateforme FoodAid

Introduction

Ce chapitre présente l'analyse des besoins, les acteurs, les fonctionnalités principales, ainsi que les choix de conception de la plateforme FoodAid. Une conception rigoureuse est essentielle pour garantir la cohérence du système et faciliter son développement.

2.1 Analyse des besoins

2.1.1 Périmètre du projet

Le périmètre de FoodAid couvre les fonctionnalités essentielles liées à la gestion des dons alimentaires, des utilisateurs et de la logistique associée. Il inclut la gestion des comptes, des dons, des missions, des rôles, les services de communication, de géolocalisation et d'assistance intelligente.

2.1.2 Acteurs de la plateforme

La plateforme FoodAid implique six acteurs principaux :

- **Administrateur** : supervision, validation, gestion des utilisateurs et statistiques ;
- **Donateur** : publication et suivi des dons, consultation des associations ;
- **Association** : réception des dons, organisation des bénéficiaires et missions ;
- **Bénéficiaire** : suivi des aides via un profil social rattaché à une association ;
- **Livreur** : transport des dons, gestion des missions et disponibilité, géolocalisation ;
- **Volontaire** : participation aux actions de terrain, missions et urgences.

2.1.3 Diagrammes des cas d'utilisation

Les diagrammes des cas d'utilisation présentent les interactions entre les six acteurs principaux de la plateforme FoodAid et les fonctionnalités clés du système.

Livreur

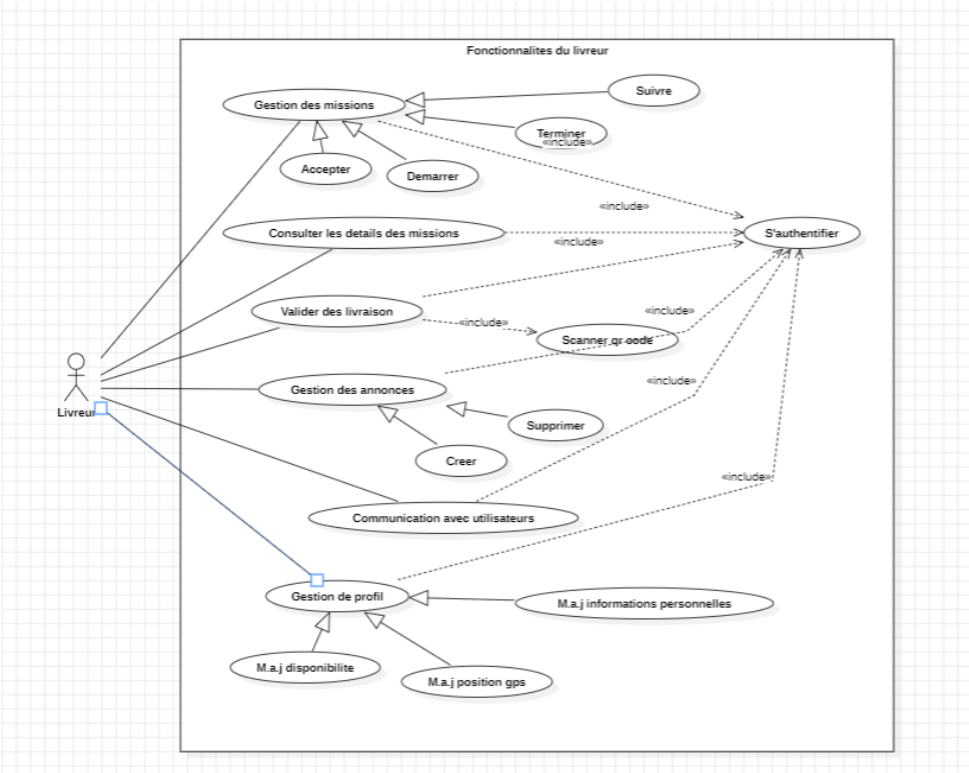


Figure 2.1 – Diagramme des cas d'utilisation du livreur

Le livreur dispose de fonctionnalités lui permettant de gérer sa présence sur le terrain et d'exécuter les missions de livraison. Il peut tout d'abord mettre à jour sa disponibilité ainsi que sa position GPS, ce qui permet au système de lui proposer les missions adaptées à sa localisation. Il peut consulter l'ensemble des missions disponibles, en accepter une, puis la démarrer et la terminer une fois la livraison effectuée. Le livreur peut également scanner un QR Code pour valider la prise en charge ou la remise d'un don. Enfin, il a la possibilité de créer des annonces, de gérer ses propres annonces et de communiquer avec les autres utilisateurs via la messagerie intégrée.

Donateur

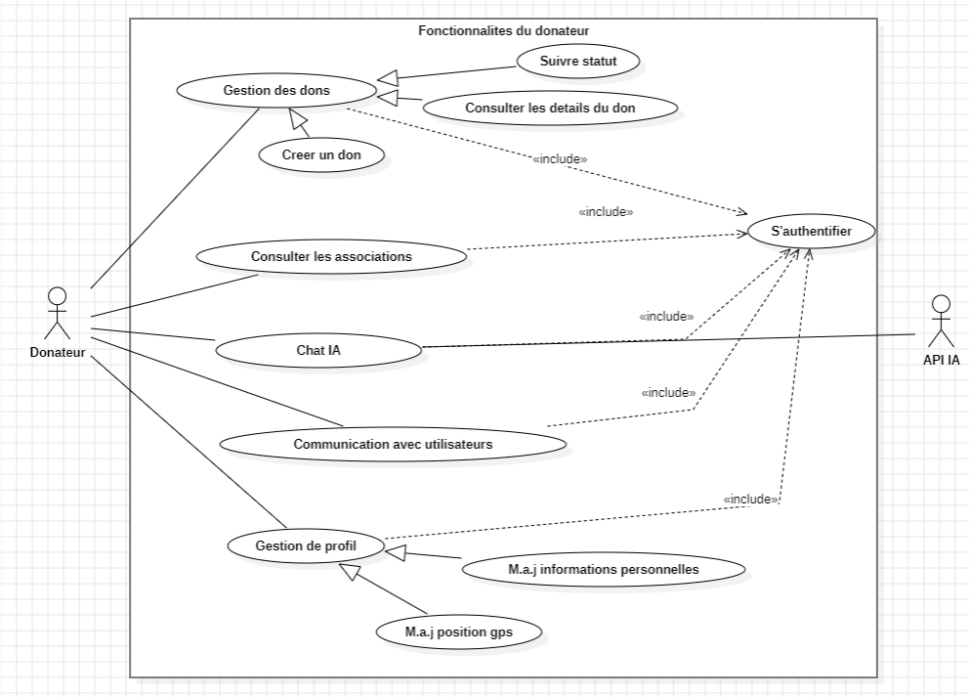


Figure 2.2 – Diagramme des cas d'utilisation du donateur

Le donateur peut s'inscrire sur la plateforme et se connecter à son espace personnel. Il a la possibilité de créer un don alimentaire en renseignant les informations nécessaires, de consulter la liste de ses dons et de suivre l'évolution de leurs statuts en temps réel. Il peut également consulter les associations partenaires afin de choisir celle à qui il souhaite confier son don. La plateforme lui propose une vue IA des besoins globaux, lui permettant d'identifier les zones prioritaires. Enfin, le donateur peut discuter avec d'autres utilisateurs et contacter le support en cas de besoin.

Administrateur

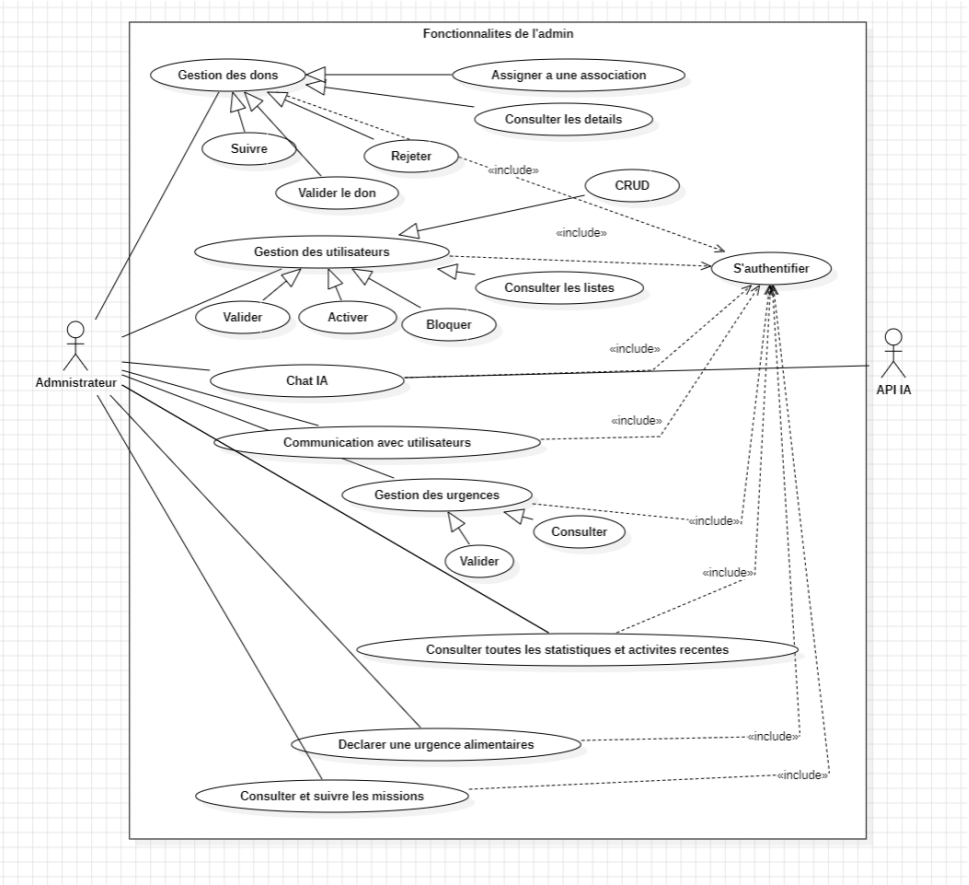


Figure 2.3 – Diagramme des cas d'utilisation de l'administrateur

L'administrateur est le superviseur de la plateforme. Il peut se connecter et consulter son tableau de bord qui lui offre une vue d'ensemble de l'activité. Il gère les utilisateurs en validant les comptes, en vérifiant les livreurs et les associations, et en activant ou bloquant les profils selon les besoins. Il valide ou rejette les dons et les assigne à une association. L'administrateur consulte les missions en cours, gère les urgences et consulte les statistiques globales. Il a également accès au rapport logistique généré par l'IA et peut participer aux échanges via le chat.

Volontaire

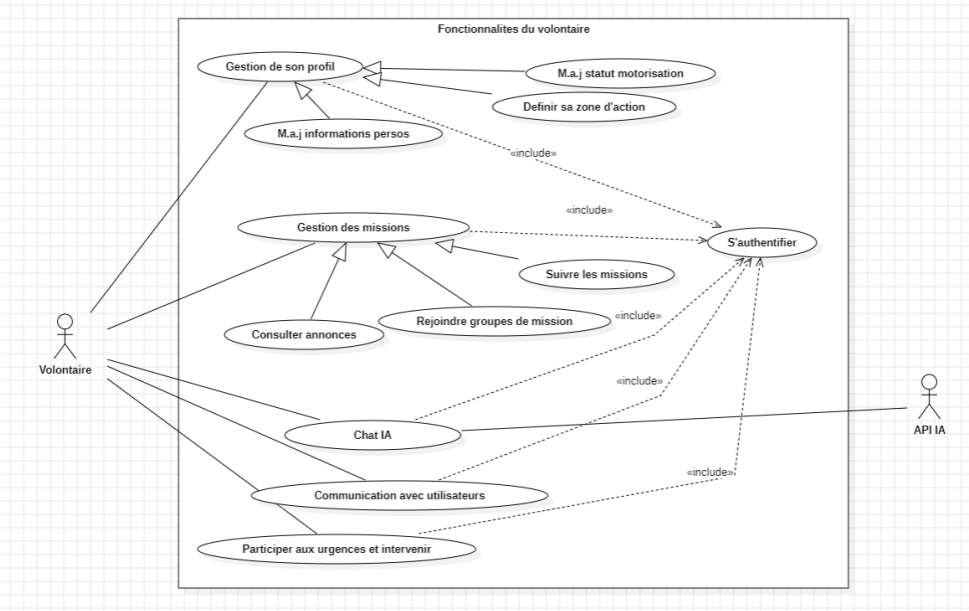


Figure 2.4 – Diagramme des cas d'utilisation du volontaire

Le volontaire peut gérer ses informations personnelles, notamment sa motorisation, ses disponibilités et sa zone d'action géographique. Il peut consulter les annonces publiées sur la plateforme et rejoindre des groupes de mission. Le système lui permet de suivre ses heures de bénévolat et de participer aux urgences pour lesquelles il a été autorisé. Ces fonctionnalités lui offrent une vue claire de ses engagements et de ses contributions.

Association

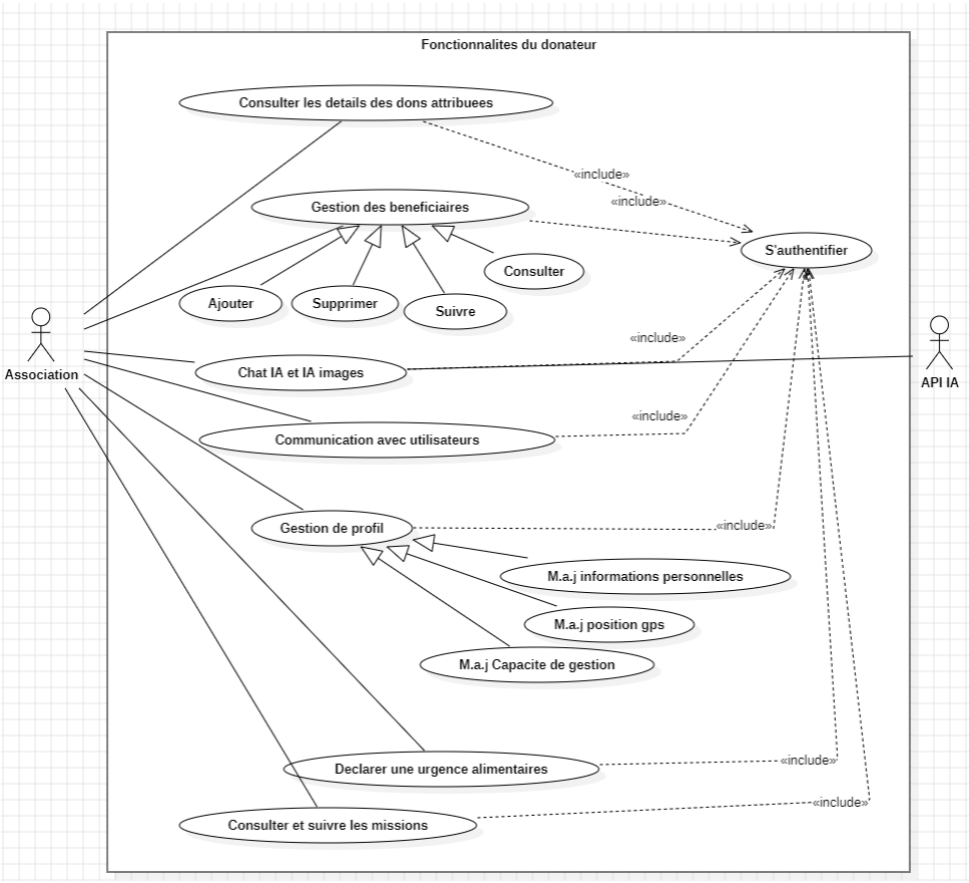


Figure 2.5 – Diagramme des cas d'utilisation de l'association

L'association peut gérer son profil et définir sa localisation exacte pour faciliter les livraisons. Elle consulte les dons qui lui sont assignés et gère les bénéficiaires rattachés à sa structure. En cas de situation critique, elle peut déclarer une urgence alimentaire. L'association utilise également les services d'intelligence artificielle pour générer des recettes à partir des ingrédients disponibles ou pour créer des images culinaires. Elle peut discuter via le chat et consulter l'assistant intelligent pour obtenir des recommandations.

Synthèse des cas d'utilisation

Le tableau ci-dessous récapitule l'ensemble des fonctionnalités accessibles à chaque acteur :

Acteur	Fonctionnalités principales
Administrateur	Gestion des utilisateurs, validation des comptes et dons, supervision des associations et livreurs, suivi des missions et urgences, statistiques, rapports IA, messagerie
Donateur	Création et suivi des dons, consultation des associations, vue IA des besoins, messagerie, support
Association	Gestion du profil et des bénéficiaires, consultation des dons reçus, déclaration d'urgences, IA recette/image, assistant, messagerie
Livreur	Gestion de la disponibilité et position GPS, gestion des missions (consultation, acceptation, exécution), validation par QR Code, annonces, messagerie
Volontaire	Gestion des disponibilités et zone d'action, consultation des annonces, participation aux urgences, suivi des heures
Bénéficiaire	Consultation du statut, réception des aides via l'association

Table 2.1 – Synthèse des cas d'utilisation par acteur

2.1.4 Diagrammes de séquence

Les diagrammes de séquence ci-dessous illustrent les interactions entre les différents composants du système pour les fonctionnalités clés, organisées par acteur.

Acteur : Administrateur

L'administrateur supervise l'ensemble de la plateforme. Il gère les utilisateurs, valide les comptes, vérifie les livreurs et associations, et supervise les missions.

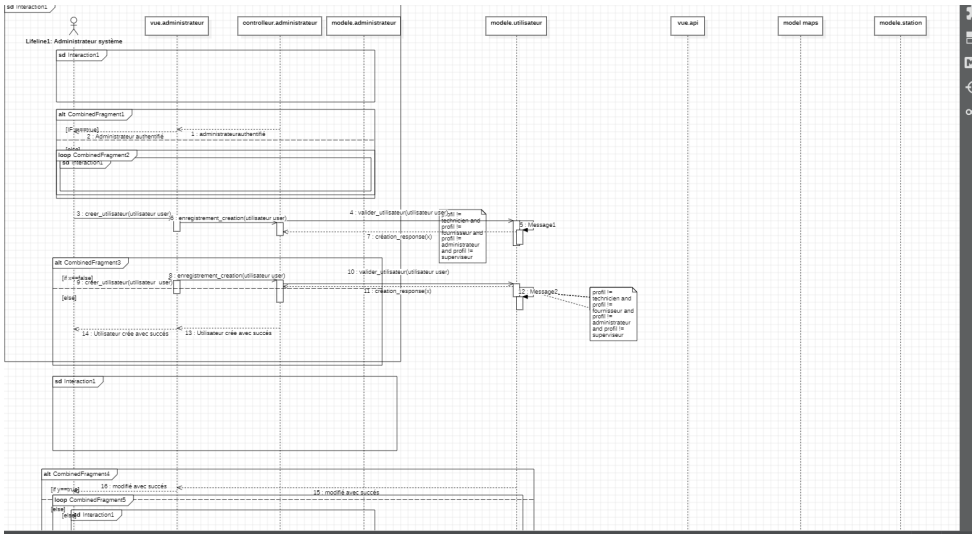


Figure 2.6 – Diagramme de séquence - Authentification de l'administrateur avec gestion des erreurs

Lors de l'authentification, l'administrateur saisit ses identifiants (email et mot de passe)

via l'interface de connexion. Le système vérifie ces informations auprès de la base de données. Si les identifiants sont corrects, un token JWT est généré et retourné à l'administrateur. Ce dernier accède alors à son tableau de bord personnalisé contenant l'ensemble des indicateurs de supervision. En cas d'erreur de connexion, le système affiche un message d'erreur explicite et propose à l'utilisateur de réessayer après correction. Après 5 tentatives échouées, le compte est temporairement verrouillé pendant 15 minutes pour prévenir les attaques par force brute. Un lien "Mot de passe oublié" permet d'initier une réinitialisation par email. En cas de perte de connexion réseau, le système met en file d'attente les requêtes et tente automatiquement de les retransmettre une fois la connexion rétablie.

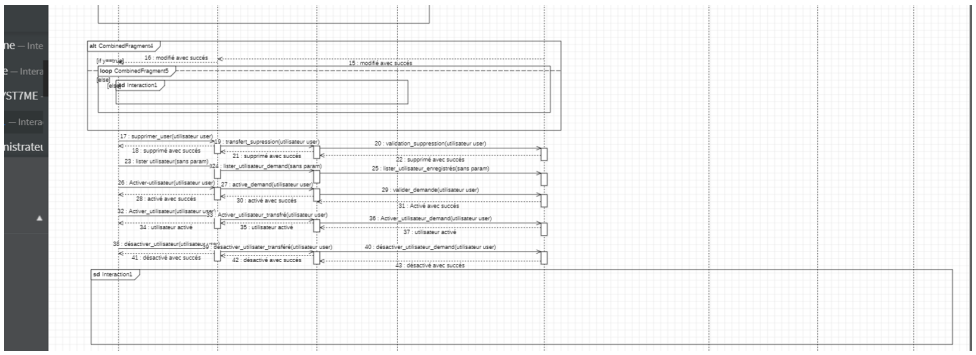


Figure 2.7 – Diagramme de séquence - Activation/désactivation/gestion et affichage de tous types des utilisateurs

L'administrateur consulte la liste complète des utilisateurs enregistrés sur la plateforme. Il peut filtrer cette liste par rôle pour faciliter la recherche. Pour chaque compte, l'administrateur peut activer, bloquer ou supprimer l'utilisateur selon les besoins. Le système enregistre automatiquement chaque action dans les logs d'audit pour assurer la traçabilité des opérations.

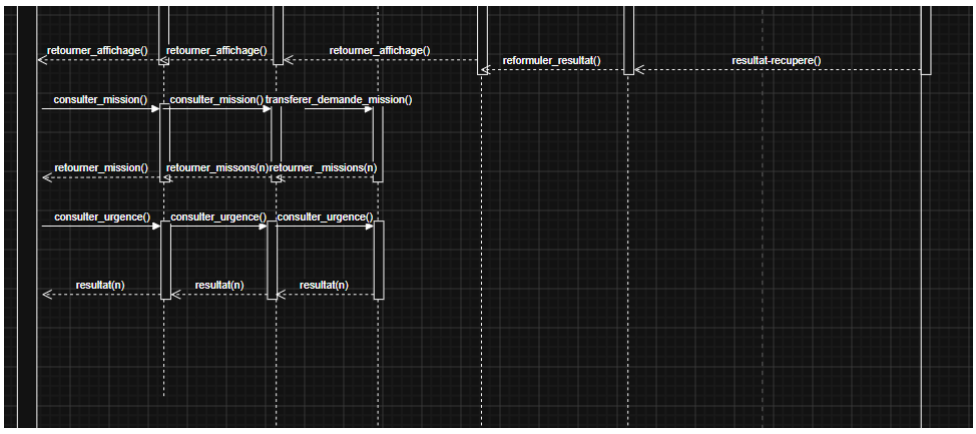


Figure 2.8 – Diagramme de séquence - Gestion des missions par l'administrateur

L'administrateur consulte la liste des missions en cours sur la plateforme. Il peut suivre l'avancement de chaque mission en temps réel et visualiser le statut actuel. En cas de

besoin, il peut réassigner une mission à un autre livreur. Le système met automatiquement à jour les statuts et notifie les parties prenantes concernées.

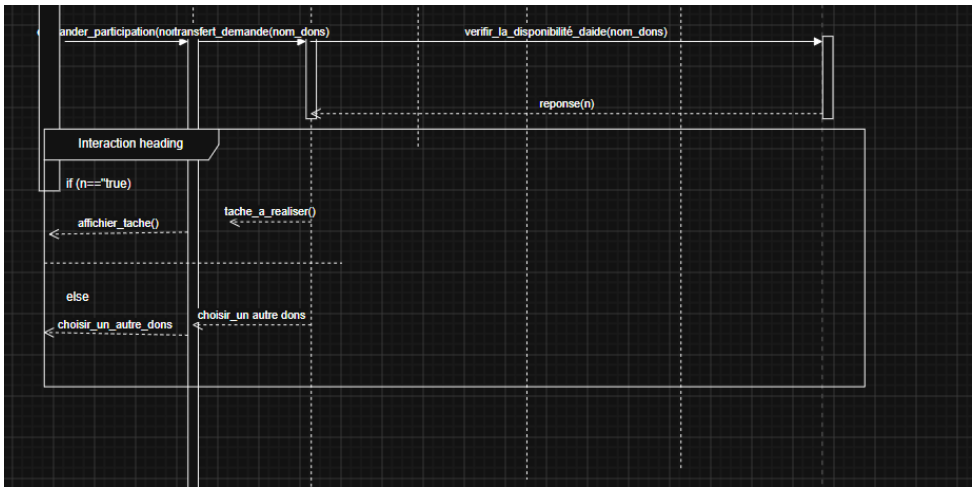


Figure 2.9 – Diagramme de séquence - Vérification des dons nécessitant des volontaires

L'administrateur consulte les demandes de vérification émanant des livreurs et des associations. Il examine attentivement les documents fournis pour s'assurer de leur conformité. Après analyse, il valide ou rejette la demande. Le statut du compte est alors mis à jour en conséquence, permettant ou non à l'utilisateur d'accéder aux fonctionnalités réservées.

Acteur : Livreur

Le livreur est responsable du transport des dons alimentaires. Il gère ses missions, met à jour sa disponibilité et valide les livraisons via QR Code.

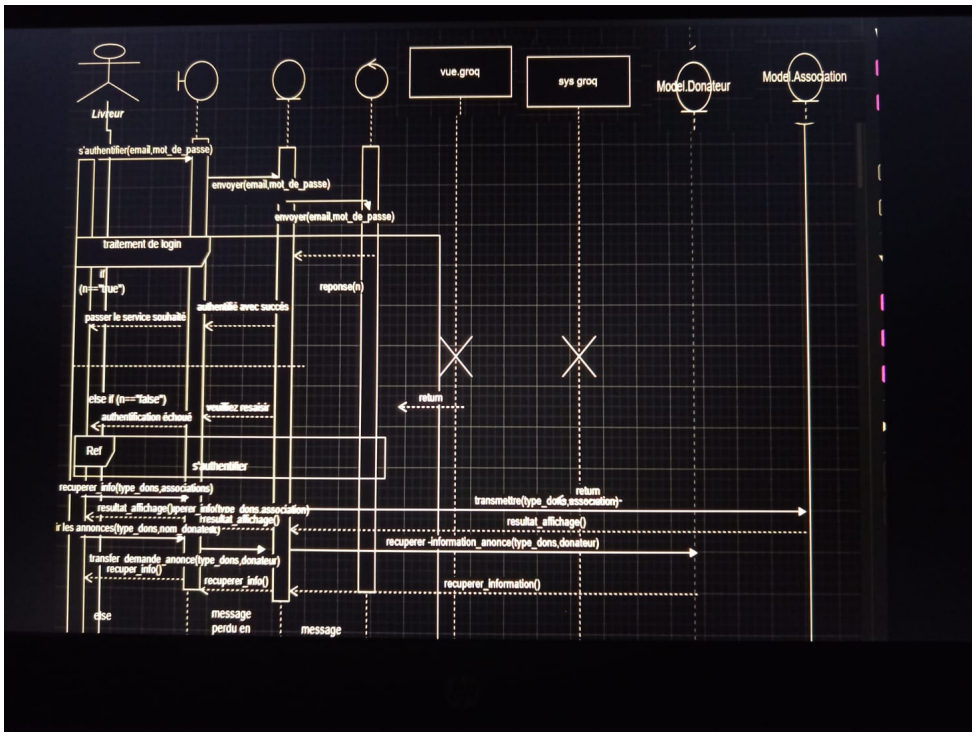


Figure 2.10 – Diagramme de séquence - Gestion des missions par le livreur

Le livreur consulte la liste des missions disponibles sur son tableau de bord. Il peut accepter une mission selon sa disponibilité et ses compétences. Une fois la mission acceptée, son statut passe à "ACCEPTED". Le livreur démarre ensuite la mission en se rendant au point de retrait. Après avoir effectué la livraison, il termine la mission, ce qui met à jour le statut global du don associé.

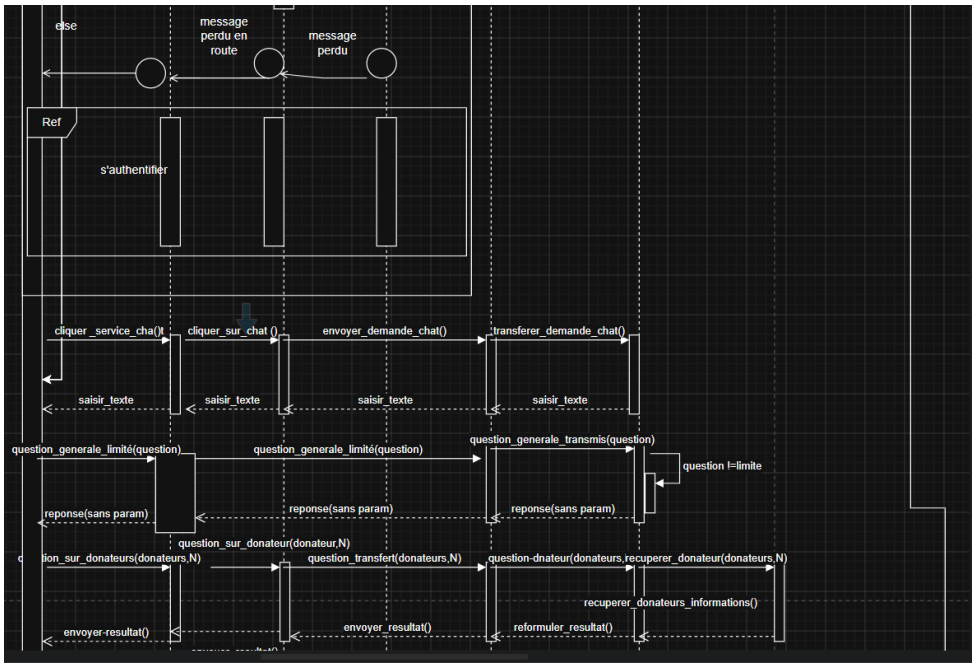


Figure 2.11 – Diagramme de séquence - Communication avec la base de données et système RAG

Le livreur interagit avec le système via l'interface utilisateur. Ses actions déclenchent des requêtes vers la base de données PostgreSQL pour récupérer ou mettre à jour les informations. Parallèlement, certaines requêtes peuvent interroger le LLM via WebClient pour obtenir des informations contextuelles ou des recommandations. Le système RAG (Retrieval-Augmented Generation) intervient en recherchant dans la base vectorielle ChromaDB les documents pertinents pour enrichir le contexte de la requête avant de l'envoyer au LLM. Les réponses sont ensuite renvoyées au livreur avec les références des sources utilisées.

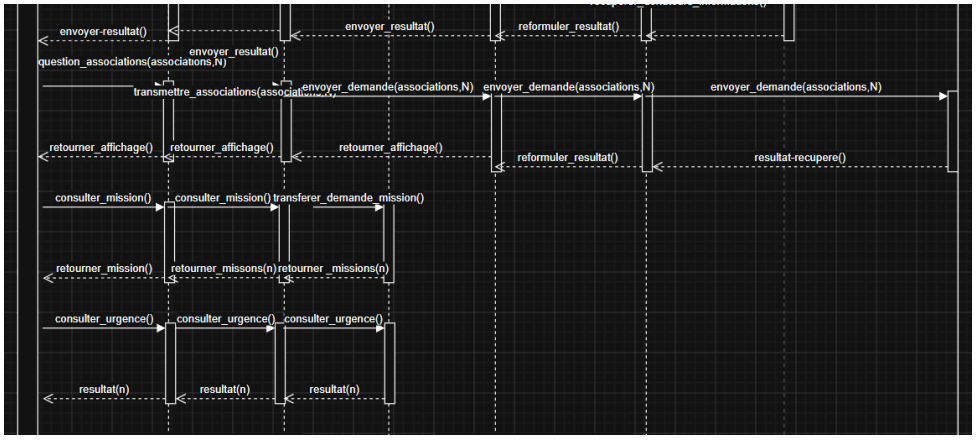


Figure 2.12 – Diagramme de séquence - Consulter des missions affectées avec un code QR de positionnement

Le livreur consulte la liste des missions qui lui ont été affectées. Pour chaque mission, un QR Code unique est généré et associé au don correspondant. Le livreur scanne ce QR

Code à l'aide de son appareil au moment de la livraison. Le backend vérifie la validité du QR Code. Si la vérification est positive, la livraison est confirmée et le statut du don est mis à jour automatiquement. Une notification est alors envoyée à l'administrateur.

Acteur : Donateur

Le donateur publie des dons alimentaires et suit leur parcours jusqu'à la distribution.

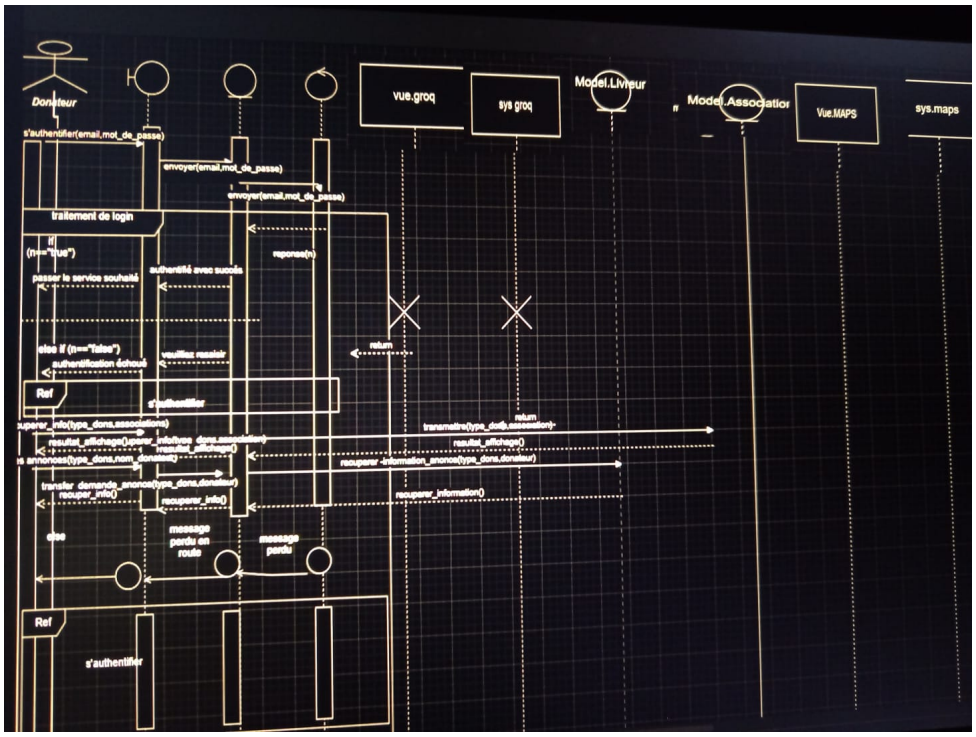


Figure 2.13 – Diagramme de séquence - Authentification du donateur avec gestion des erreurs

Le donateur saisit ses identifiants de connexion. Le système vérifie la validité des informations d'authentification. Si les identifiants sont corrects, un token JWT est généré et retourné. Le donateur accède alors à son tableau de bord personnalisé. En cas d'identifiants incorrects, un message d'erreur s'affiche invitant à ressaisir les informations. Le système gère également les pertes de connexion en mettant les requêtes en file d'attente et en tentant automatiquement de les retransmettre lors du rétablissement de la connexion.

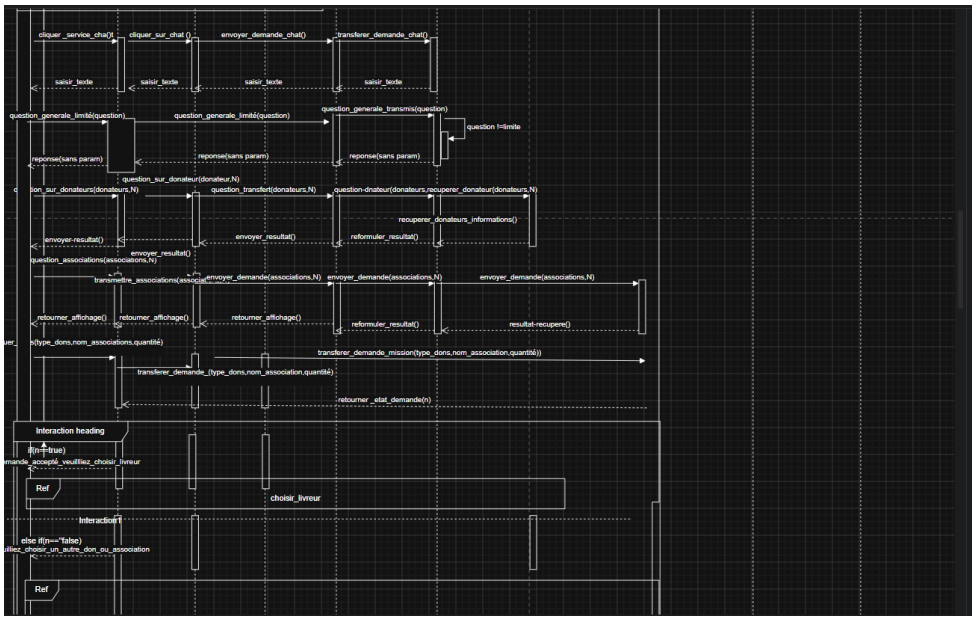


Figure 2.14 – Diagramme de séquence - Consultation de la vue IA des besoins globaux

Le donateur accède à la vue IA des besoins alimentaires. Le système analyse les données collectées à partir des dons publiés et des demandes exprimées par les associations. Une visualisation cartographique des besoins par zone géographique est générée et affichée. Le donateur peut ainsi identifier les zones prioritaires.

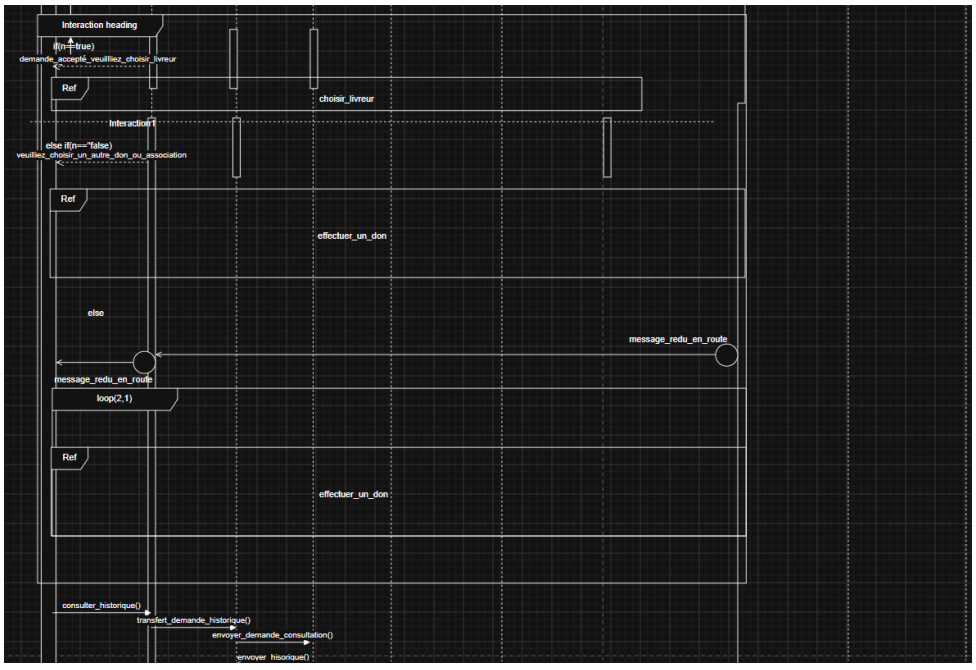


Figure 2.15 – Diagramme de séquence - Communication avec le support

Le donateur envoie un message via le chat de support intégré. Le message est transmis à l'administrateur responsable. L'administrateur peut répondre en temps réel. Le donateur reçoit la réponse instantanément via l'interface de chat.

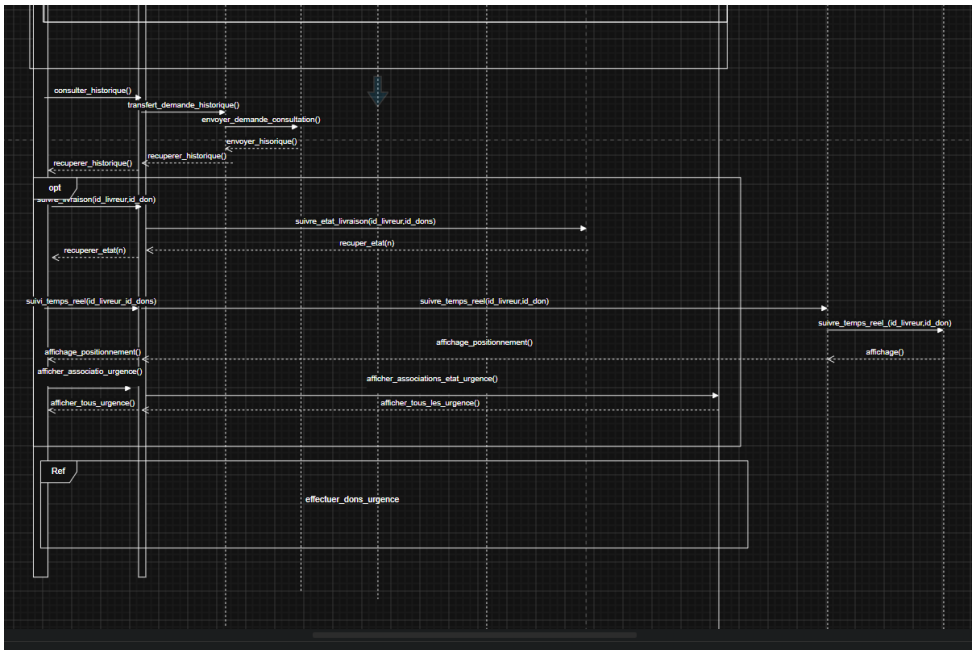


Figure 2.16 – Diagramme de séquence - Historique des dons et le suivi en temps réel de livreur concerné

Le donateur consulte l'historique complet de ses dons sur la plateforme. Pour chaque don, il peut suivre l'évolution du statut en temps réel. Le système lui permet également de visualiser la position GPS du livreur chargé de la livraison via une carte interactive.

Acteur : Association

L'association reçoit et distribue les dons alimentaires. Elle gère les bénéficiaires, déclare les urgences et utilise les services IA.

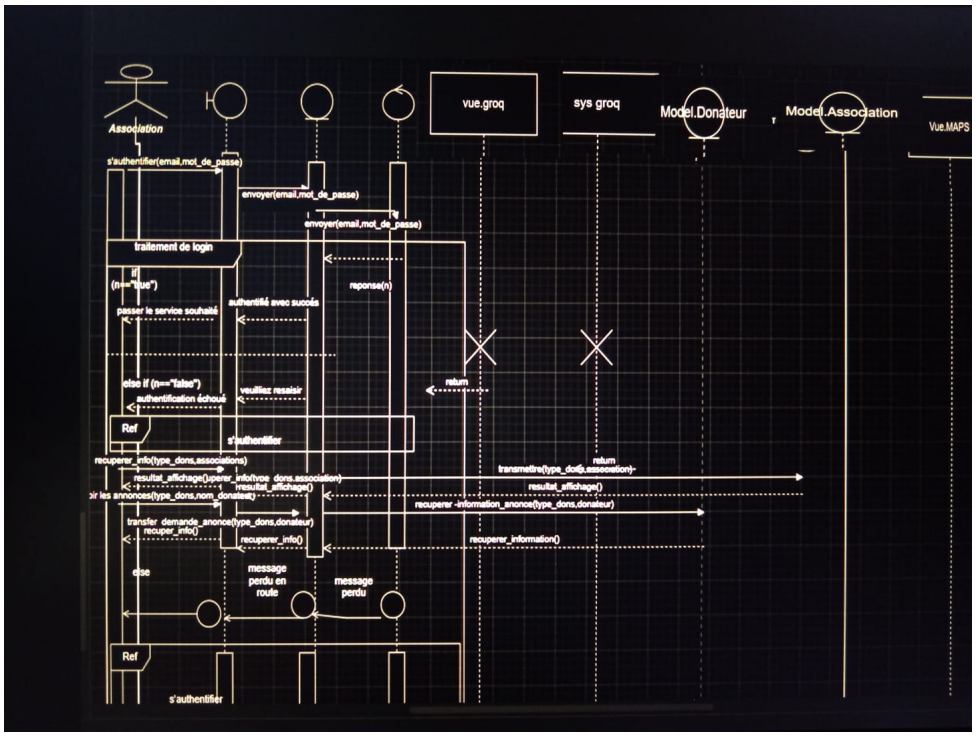


Figure 2.17 – Diagramme de séquence - Authentification de l'association avec gestion des erreurs

L'association se connecte à la plateforme en saisissant ses identifiants. Le système vérifie les informations d'authentification et génère un token JWT en cas de succès. L'association accède alors à son tableau de bord dédié. En cas d'échec de connexion, le système propose de réessayer et gère les pertes de réseau en retransmettant automatiquement les requêtes lors du rétablissement de la connexion.

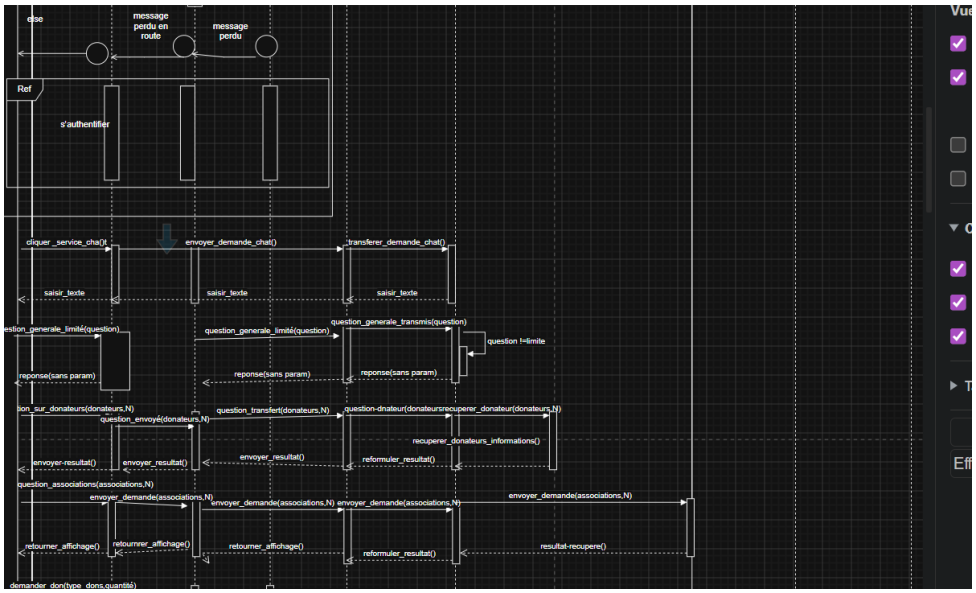


Figure 2.18 – Diagramme de séquence - Gestion des bénéficiaires

L'association consulte la liste complète des bénéficiaires rattachés à sa structure. Elle peut ajouter un nouveau bénéficiaire, modifier ses informations ou supprimer son profil. L'association suit également la distribution des aides à chaque bénéficiaire.

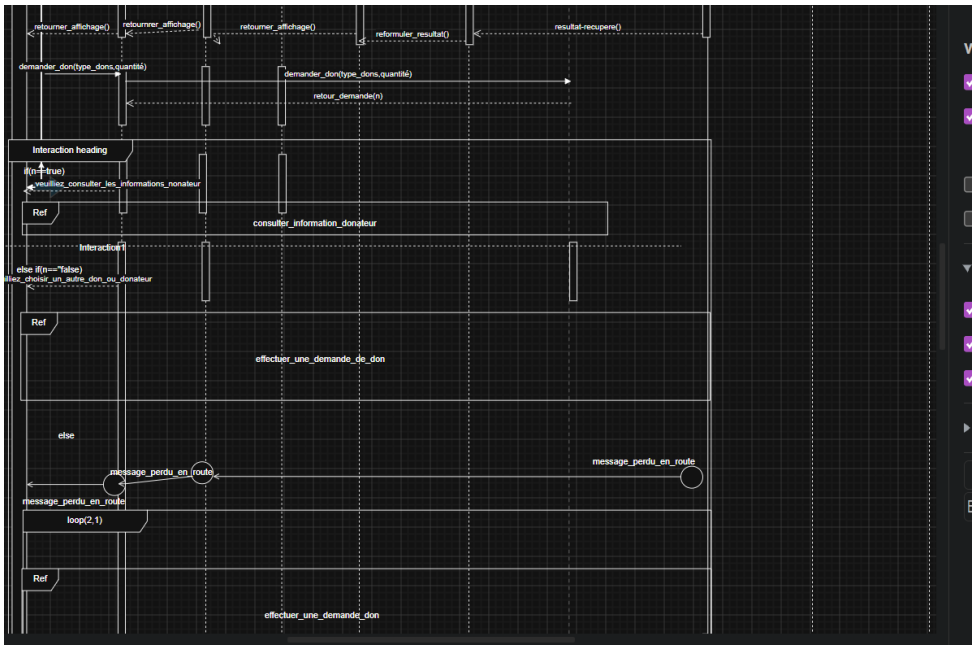


Figure 2.19 – Diagramme de séquence - Déclaration de demande de dons

L'association identifie ses besoins alimentaires et remplit le formulaire de déclaration de demande de dons. Elle précise les quantités nécessaires, le type d'aliments et la date souhaitée. La demande est enregistrée et devient visible pour les donateurs potentiels.

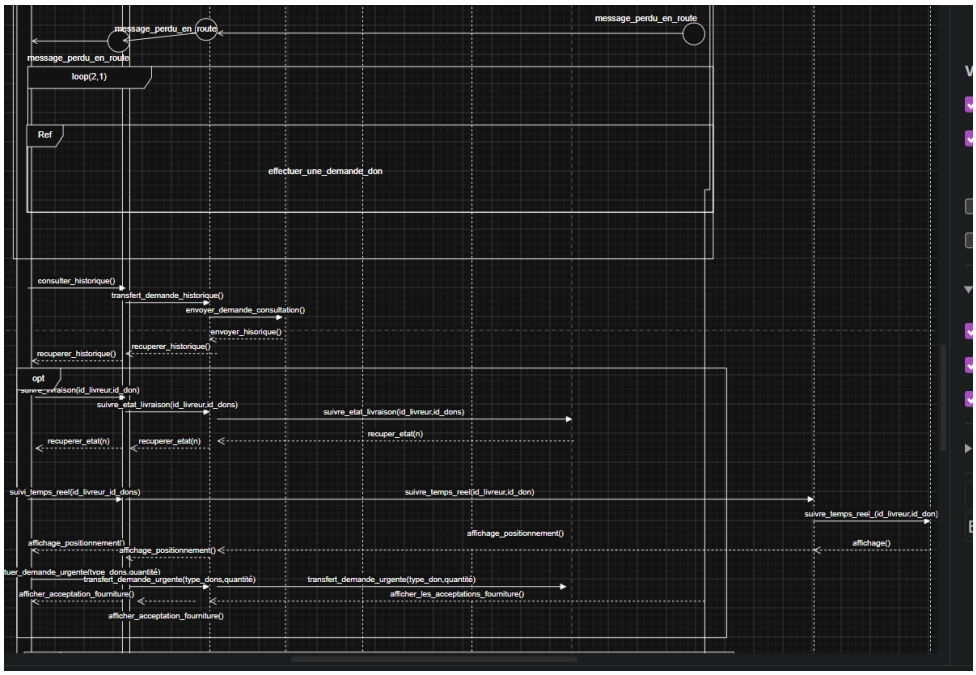


Figure 2.20 – Diagramme de séquence - Suivi en temps réel de livreur et demande historique

L'association peut suivre en temps réel la position du livreur chargé de la livraison d'un don. Le système affiche la localisation GPS sur une carte interactive. L'association peut également consulter l'historique complet des dons reçus.

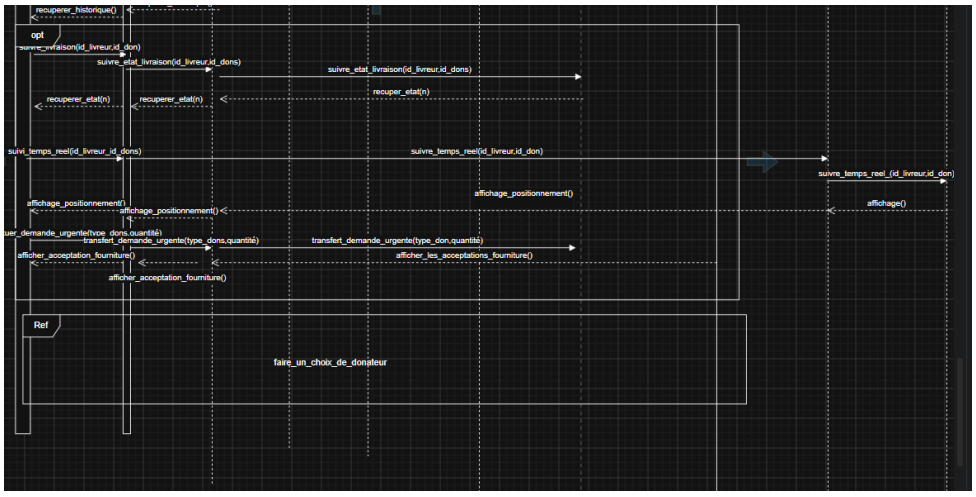


Figure 2.21 – Diagramme de séquence - Urgence et suivi des dons

L'association identifie une situation d'urgence alimentaire et remplit le formulaire de déclaration. Elle définit le niveau de criticité et précise la localisation exacte. L'urgence est enregistrée et visible immédiatement par l'administrateur. Une notification est envoyée automatiquement aux volontaires disponibles.

2.1.5 Diagramme de classes

Le diagramme de classes présenté ci-dessous décrit la structure statique du système, en montrant les principales entités et leurs relations.

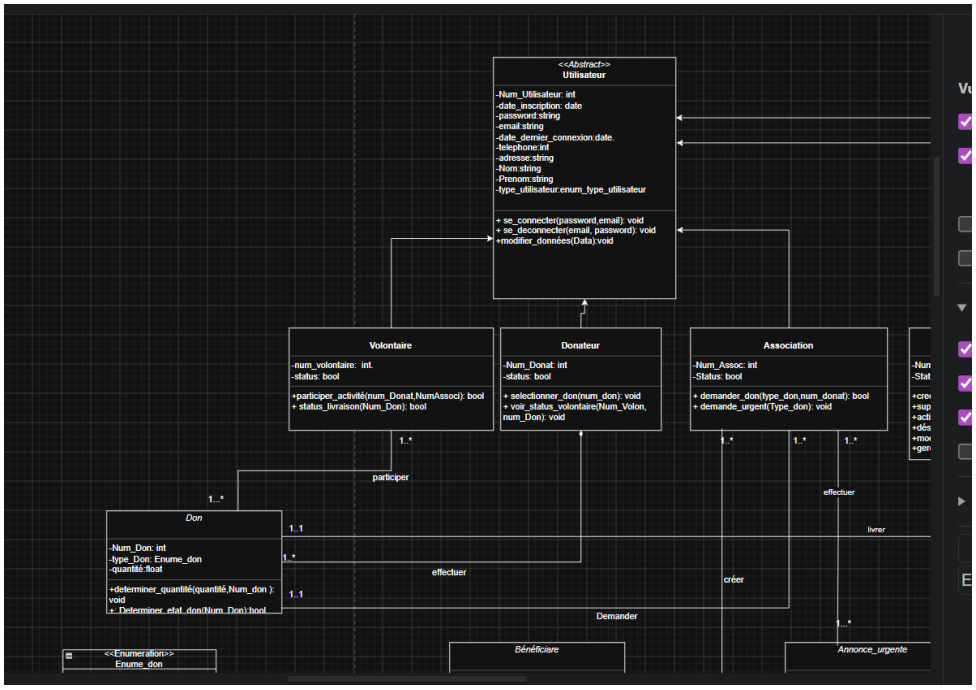


Figure 2.22 – Diagramme de classes de la plateforme FoodAid (partie 1)

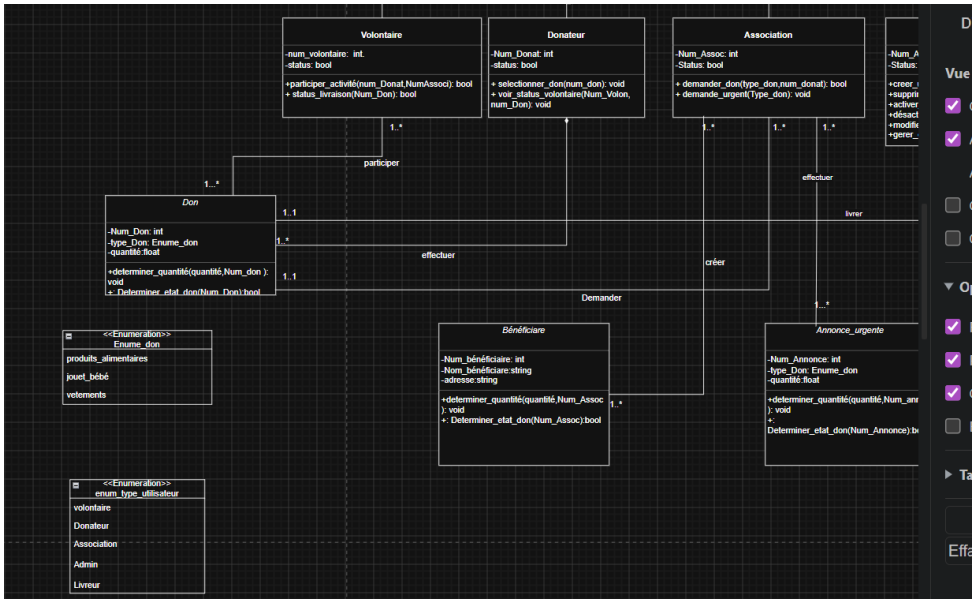


Figure 2.23 – Diagramme de classes de la plateforme FoodAid (partie 2)

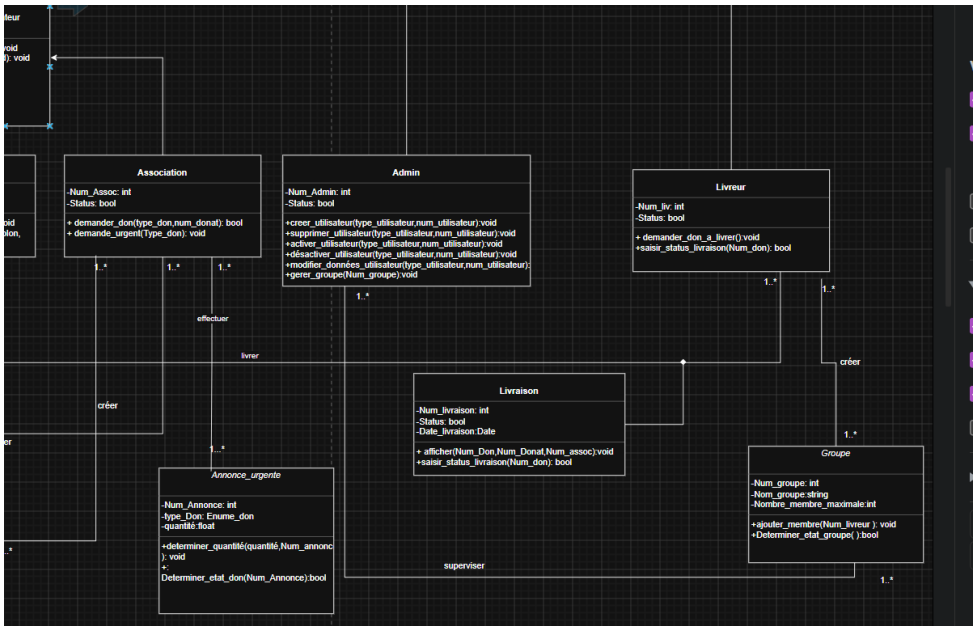


Figure 2.24 – Diagramme de classes de la plateforme FoodAid (partie 3)

Les classes principales sont :

- **Utilisateur** : utilisateur central avec ses informations communes ;
- **Volontaire, Association, Livreur, Donateur, Beneficiaire, Admin** : profils spécialisés ;
- **Don, Mission, GroupeMission** : éléments du flux de dons et livraisons ;
- **AnnonceUrgente, Livraison, Groupe, Aide** : gestion associative et transverse ;

2.2 Conception de la base de données

2.2.1 Modèle conceptuel

Le modèle repose sur une table centrale `users` avec des profils spécialisés selon le rôle (`donateur_profiles`, `association_profiles`, `livreur_profiles`, `volontaire_profiles`, `beneficiaire_profiles`). Les principales entités et leurs relations sont :

Table 2.2 – Principales relations de la base de données

Entité source	Relation	Entité cible
users	1-1	*_profiles (5 tables)
donateur_profiles	1-n	dons
association_profiles	1-n	dons, beneficiaire_profiles, urgences_extremes

dons	1-1	missions
livreur_profiles	1-n	missions
missions	1-1	groupes_mission
groupes_mission	n-n	volontaire_profiles
users	n-n	chat_rooms
chat_rooms	1-n	chat_messages

2.2.2 Liste des tables de la base de données

La base de données FoodAid est composée des 21 tables suivantes, organisées par catégorie fonctionnelle :

Table 2.3 – Liste complète des tables de la base de données FoodAid

Nom de la table	**Description**
1. Tables de gestion des utilisateurs	
users	Table centrale des comptes utilisateurs (authentification, email, mot de passe, rôle, statut, localisation)
donateur_profiles	Profil spécifique aux donateurs
association_profiles	Profil spécifique aux associations (vérification, documents)
livreur_profiles	Profil spécifique aux livreurs (vérification, disponibilité, motorisation)
volontaire_profiles	Profil spécifique aux volontaires (disponibilités, zone d'action)
beneficiaire_profiles	Profil spécifique aux bénéficiaires
2. Tables de gestion des dons et missions	
dons	Gestion des offres de dons alimentaires (type, quantité, date péremption, statut, GPS retrait)
missions	Suivi des missions de livraison (trajet, statut, livreur assigné, météo)
groupes_mission	Groupement de plusieurs missions
urgences_extremes	Déclarations d'urgences alimentaires (niveau criticité, localisation)
3. Tables de communication	
chat_rooms	Salons de discussion (privés, groupes, support)
chat_messages	Messages échangés dans les salons
notifications	Notifications envoyées aux utilisateurs
4. Tables de validation et suivi	

Nom de la table	Description
qr_codes	Codes QR générés pour les dons (code unique, date expiration, statut scanné)
annonces	Annonces publiées par les livreurs
5. Tables IA et RAG	
recettes	Recettes générées par l'IA à partir des ingrédients
ingredients	Ingrédients utilisés dans les recettes
rag_documents	Documents indexés pour le système RAG (PDF, guides, rapports)
6. Tables associatives et gestion des associations	
stocks	Gestion des stocks des associations
besoins	Besoins alimentaires exprimés par les associations
aides	Aides distribuées aux bénéficiaires

2.2.3 Tables principales

Les tables principales de la base de données gèrent les fonctionnalités suivantes :

- **Users** : table centrale gérant l'authentification, les rôles, la localisation et le statut des utilisateurs ;
- **Profils** : cinq tables distinctes stockant les données spécifiques à chaque rôle (`donateur_profiles`, `association_profiles`, `livreur_profiles`, `volontaire_profiles`, `beneficiaire_profiles`) ;
- **Dons** : gestion des offres alimentaires incluant le type, la quantité, la date de péremption, le statut et les coordonnées GPS du point de retrait ;
- **Missions** : suivi des trajets de livraison avec le statut, le livreur assigné et les conditions météorologiques ;
- **Urgences, stocks, besoins, aides** : gestion associative pour les situations critiques et la planification des distributions ;
- **Notifications, QR Codes, chat** : fonctionnalités transverses assurant la communication et la traçabilité ;
- **Tables IA** : stockage des ingrédients, recettes générées et données RAG pour les services d'intelligence artificielle.

3 Réalisation et interfaces de l'application

Introduction

Ce chapitre présente l'environnement de développement, l'architecture technique, la structure du projet, les modules implémentés, les interfaces et les tests.

3.1 Environnement de développement

3.1.1 Environnement matériel

Le projet est développé sur un poste local capable d'exécuter simultanément :

- Un serveur backend Spring Boot ;
- Un serveur frontend Vite ;
- Une base PostgreSQL ;
- Les outils de build Maven et npm.

3.1.2 Environnement logiciel

L'environnement logiciel comprend :

- Java 17 ;
- Spring Boot 3.5.0 ;
- Maven ;
- Node.js et npm ;
- React 18 ;
- Vite 5 ;
- PostgreSQL ;
- Flyway ;
- Navigateur web moderne ;
- Swagger/OpenAPI pour consulter les endpoints.

3.2 Description des besoins techniques

3.2.1 Architecture générale de l'application

L'architecture est de type client-serveur :

— le navigateur charge l'application React ;

— React communique avec Spring Boot via Axios ;

— Spring Boot expose des API REST et WebSocket ;

— PostgreSQL stocke les données ;

— Flyway gère les migrations ;

— Groq et les services IA sont appelés par WebClient ;

— les fichiers PDF du RAG local sont lus depuis `_archive/chatbot-legacy/data`.

3.2.2 Technologies utilisées

Couche	**Technologies**
Frontend	React 18, Vite, Tailwind CSS, React Router DOM, Axios, Zustand
UI et visualisation	Lucide React, Heroicons, Recharts, Chart.js, React-Chartjs-2, Framer Motion
Cartographie	Leaflet, React-Leaflet, Leaflet Heat
QR Code frontend	html5-qrcode
Backend	Java 17, Spring Boot 3.5, Spring Web, Spring Security, Spring Data JPA
Base de données	PostgreSQL, Flyway
Temps réel	Spring WebSocket
Sécurité	JWT, BCrypt, Spring Security
IA	Groq, WebClient, RAG local, ChromaDB en secours
PDF	Apache PDFBox
Documentation API	Springdoc OpenAPI
Tests	Vitest, Spring Boot Test, Spring Security Test, Testcontainers

Table 3.1 – Technologies utilisées par couche

Front-end : React.js

Le frontend est une application React/Vite. Il contient des pages par domaine : authentification, dashboards, dons, missions, urgences, associations, bénéficiaires, volontaires,

livreurs, annonces, chat, profil et administration.

Back-end : Spring Boot

Le backend est un service Spring Boot unifié. Il expose les APIs REST, gère la sécurité, les transactions, les entités JPA, les notifications, le QR Code, les services IA et la communication avec la base de données.

Base de données : PostgreSQL

La base de données de développement est `foodaid_dev`. Les migrations Flyway créent les tables principales, les index, les données de test, les tables chat, annonces, recettes, données RAG et champs GPS.

Authentification et sécurité : JWT

L'authentification passe par `/api/auth/login`, `/api/auth/register`, `/api/auth/refresh` et `/api/auth/logout`. Les tokens JWT contiennent l'identité et le rôle de l'utilisateur. Les routes sont protégées selon les rôles définis dans `SecurityConfig`.

3.2.3 Gestion des erreurs d'authentification et alternatives

La plateforme FoodAid intègre un système robuste de gestion des erreurs d'authentification pour offrir une expérience utilisateur fluide :

— **Login incorrect** : Lorsque l'utilisateur saisit un email ou un mot de passe invalide, le système affiche un message d'erreur explicite : *"Email ou mot de passe incorrect. Veuillez réessayer."* Ce message est générique pour des raisons de sécurité, afin de ne pas révéler si l'email existe ou non dans la base de données.

— **Tentatives de connexion successives** : Après 5 tentatives de connexion échouées, le compte de l'utilisateur est temporairement verrouillé pendant 15 minutes. Un message d'alerte est affiché : *"Trop de tentatives de connexion. Veuillez réessayer dans 15 minutes."* Cette mesure prévient les attaques par force brute.

— **Réinitialisation du mot de passe** : L'utilisateur peut cliquer sur le lien *"Mot de passe oublié ?"* pour recevoir un email de réinitialisation contenant un token sécurisé valable 24 heures. Le système envoie un email avec un lien unique permettant de définir un nouveau mot de passe.

— **Rafraîchissement du token JWT** : Lorsque le token JWT expire, le système tente automatiquement de le rafraîchir via l'endpoint `/api/auth/refresh`. Si le refresh token est valide, un nouveau token est généré sans que l'utilisateur ait à se reconnecter.

— **Session expirée** : Si le token est invalide ou expiré et que le rafraîchissement échoue, l'utilisateur est redirigé vers la page de connexion avec un message : *"Votre session a expiré. Veuillez vous reconnecter."*

— **Demande de resaisie du login** : En cas d'erreur de connexion, la page propose :

  — Un bouton *"Réessayer"* pour soumettre à nouveau les identifiants après correction ;

  — Un lien *"Mot de passe oublié"* pour initier une réinitialisation ;

  — Un lien *"Créer un compte"* pour les nouveaux utilisateurs ;

  — Les champs du formulaire sont automatiquement vidés partiellement pour permettre une resaisie complète et correcte.

3.2.4 Gestion de la perte de messages et alternatives réseau

La plateforme met en place plusieurs mécanismes pour gérer les perturbations réseau et les pertes potentielles de messages :

— **Timeout des requêtes** : Chaque requête HTTP est configurée avec un timeout de 30 secondes. Au-delà de ce délai, une erreur est affichée et l'utilisateur peut réessayer manuellement.

— **File d'attente des requêtes** : En cas d'échec de connexion, les requêtes API sont mises en file d'attente localement. Une fois la connexion rétablie, elles sont automatiquement retransmises.

— **Système de retry automatique** : Les requêtes échouées (erreurs 5xx, timeout) sont automatiquement réessayées jusqu'à 3 fois avec un délai exponentiel (1s, 2s, 4s) avant d'afficher une erreur à l'utilisateur.

— **Stockage local des données** : Les informations de session (token, préférences utilisateur) sont persistées dans le localStorage du navigateur. En cas de perte de connexion, l'utilisateur peut consulter ses données en mode hors ligne.

— **Reconnexion automatique au WebSocket** : Pour la messagerie en temps réel, le client WebSocket tente automatiquement de se reconnecter toutes les 5 secondes en cas de déconnexion. Les messages non envoyés sont mis en file d'attente et retransmis à la reconnexion.

— **Message de statut de connexion** : Une barre de statut permanente affiche en temps réel l'état de la connexion (connecté, reconnexion en cours, hors ligne). L'utilisateur est ainsi informé de la situation.

— **Notification de reprise** : Lorsque la connexion est rétablie, une notification toast s'affiche : *"Connexion rétablie. Les données ont été synchronisées."*

3.2.5 Intégration du chatbot avec système RAG

Présentation du système RAG

Le chatbot FoodAid intègre un système RAG (Retrieval-Augmented Generation) qui combine la recherche d'informations contextuelles avec la génération de réponses par un LLM. Cette approche permet d'obtenir des réponses précises et contextualisées basées sur les données spécifiques de la plateforme.
L'architecture RAG mise en œuvre comprend les composants suivants :

— **Base de connaissances** : Documents PDF, rapports logistiques, guides utilisateur, politiques et informations sur les dons alimentaires stockés dans le répertoire `_archive/chatbot-legacy/data`.

— **Système d'indexation** : Les documents sont découpés en chunks (segments de texte) et indexés via ChromaDB. Chaque chunk est associé à un embedding vectoriel pour permettre une recherche sémantique efficace.

— **Moteur de recherche vectorielle** : La requête de l'utilisateur est convertie en embedding vectoriel. Le système recherche les chunks les plus pertinents dans la base vectorielle en utilisant une mesure de similarité (cosine similarity).

— **Générateur de réponses** : Les chunks pertinents sont injectés dans le prompt du LLM (Groq) comme contexte. Le LLM génère une réponse naturelle en se basant sur ce contexte, garantissant des réponses factuelles et spécifiques à FoodAid.

Fonctionnement du chatbot RAG

Le flux de traitement d'une requête utilisateur suit les étapes suivantes :

1. L'utilisateur pose une question via l'interface du chatbot (ex : "Comment créer un don ?") ;
2. Le système transforme la question en embedding vectoriel via l'API d'embeddings de Groq ;
3. La base vectorielle ChromaDB est interrogée pour trouver les chunks les plus similaires à la question posée ;
4. Les 5 chunks les plus pertinents sont extraits avec leurs métadonnées (source, section, page) ;
5. Le prompt est construit en intégrant ces chunks comme contexte documentaire ;
6. Le LLM Groq génère une réponse en se basant uniquement sur le contexte fourni ;
7. La réponse est affichée à l'utilisateur avec les références des sources utilisées.

Ce système garantit que les réponses du chatbot sont toujours basées sur des informations fiables et spécifiques à la plateforme FoodAid, évitant les hallucinations du LLM.

3.2.6 WebSocket pour le suivi en temps réel

La configuration Vite proxifie `/ws` vers le backend. Les WebSockets sont utilisés pour les échanges temps réel, notamment la messagerie.

3.2.7 Outils de développement

Les outils principaux sont :

— Maven pour le backend ;

— npm pour le frontend ;

— Vite pour le serveur de développement frontend ;

— Flyway pour les migrations ;

— OpenAPI Swagger pour la documentation API ;

— logs Spring Boot pour le diagnostic ;

— scripts de build `mvn compile` et `npm run build`.

3.2.8 Structure du projet

La structure principale est :

```
FoodAid/
 backend/
    src/main/java/com/foodaid/
        controller/
        service/
        repository/
        model/
        dto/
        mapper/
        security/
        chatbot/
        recipes/
    src/main/resources/
        application.yml
        application-dev.yml
        db/migration/
 frontend/
    src/
        pages/
```

```
        services/
        components/
        router/
        config/
        store/
        utils/
docs/
```

3.3 Implémentation de la plateforme FoodAid

3.3.1 Mise en place du back-end

Le backend utilise Spring Boot avec une architecture en couches. Les contrôleurs exposent les endpoints REST, les services portent les règles métier, les dépôts gèrent l'accès aux données et les entités représentent les tables.

Les principaux contrôleurs sont :

- AuthController ;
- UserController ;
- DonController ;
- AssociationController ;
- LivreurController ;
- VolontaireController ;
- BeneficiaireController ;
- MissionController ;
- UrgenceController ;
- QrCodeController ;
- DashboardController ;
- ChatController ;
- GroupeMissionController ;
- AnnonceLivreurController ;
- AssociationAiController ;
- DonateurAiController ;
- Contrôleurs IA pour recettes, prompt photo, chat et ingestion RAG.

3.3.2 Mise en place du front-end

Le frontend React est organisé autour de routes protégées. Le fichier `router/index.jsx` définit les routes principales :

Route	**Interface**
/login	Connexion
/register	Inscription
/dashboard	Redirection selon rôle
/dashboard/admin	Tableau de bord administrateur
/dashboard/donateur	Tableau de bord donateur
/dashboard/association	Tableau de bord association
/dashboard/livreur	Tableau de bord livreur
/dashboard/volontaire	Tableau de bord volontaire
/dashboard/beneficiaire	Tableau de bord bénéficiaire
/dons	Liste des dons
/dons/creer	Création d'un don
/missions	Liste des missions
/missions/groupes	Groupes de mission
/urgences	Urgences
/associations	Associations
/associations/ai/images	IA recettes et images
/assistant	Assistant FoodAid
/admin/users	Gestion utilisateurs
/admin/ai/logistics	Rapport logistique IA
/livreur/annonces	Annonces du livreur
/annonces	Annonces visibles aux volontaires et admins

3.3.3 Mise en place de l'authentification

L'authentification repose sur JWT. Les endpoints publics sont :

— POST /api/auth/login ;

— POST /api/auth/register ;

— POST /api/auth/refresh ;

— POST /api/auth/logout.

Après connexion, le frontend stocke le token et l'utilise dans les en-têtes Authorization des appels API.
Le frontend protège les routes avec `ProtectedRoute`. Le backend protège les endpoints avec Spring Security et `@PreAuthorize`.

3.3.4 Intégration du QR Code

Le QR Code est généré sous forme d'image base64. Il est lié à un don, possède un code unique, une date d'expiration et un statut scanné/non scanné. Le scan met à jour le statut du don et envoie une notification aux administrateurs.

3.3.5 Intégration du suivi en temps réel

Le projet contient une configuration WebSocket et des composants de chat. Le frontend proxifie /ws vers le backend. La messagerie permet les salons privés, le support et les groupes de mission.

3.4 Présentation des interfaces de l'application

Captures d'écran de l'application

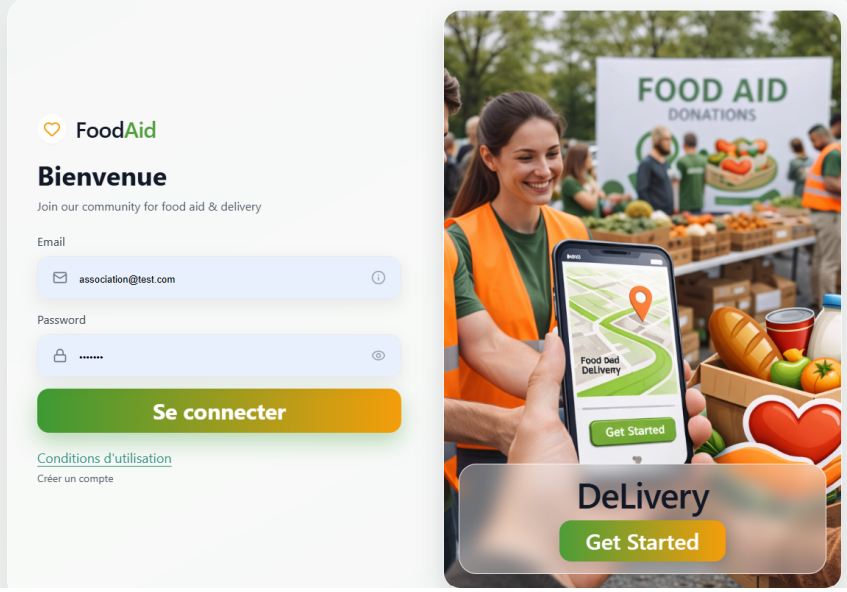


Figure 3.1 – Page de connexion de la plateforme FoodAid

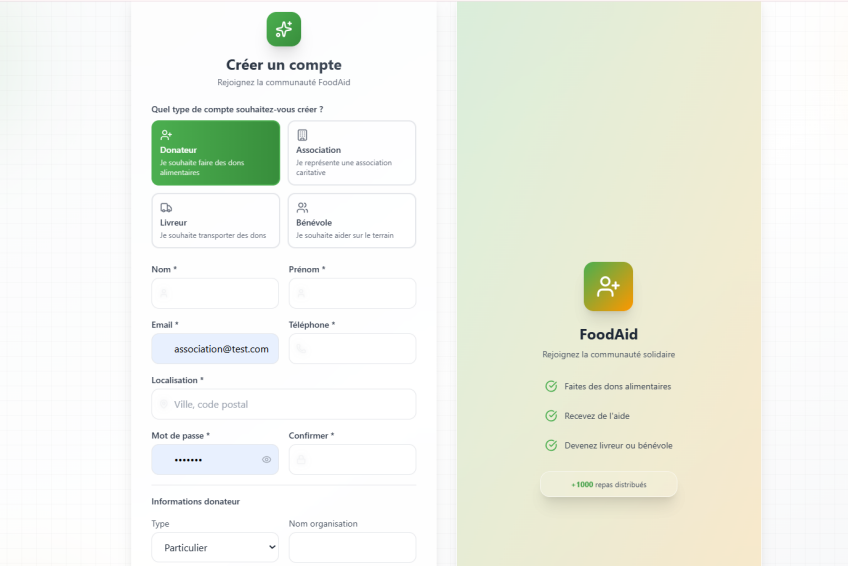


Figure 3.2 – Page d'inscription avec choix du rôle

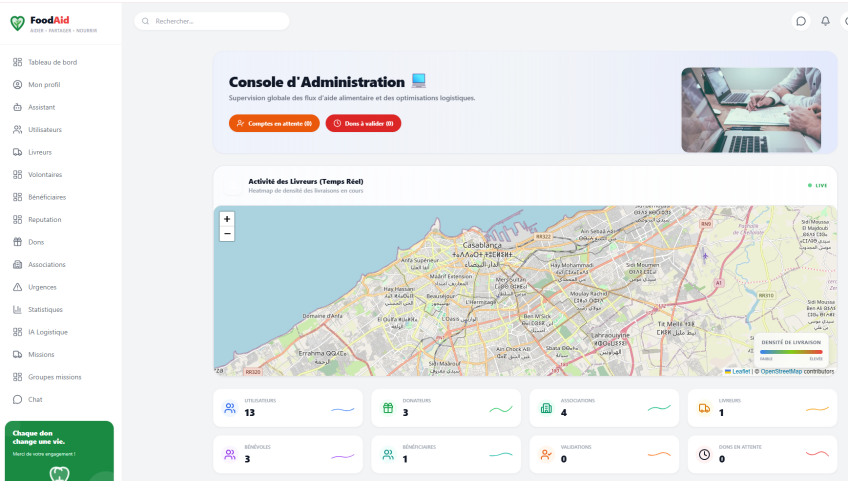


Figure 3.3 – Tableau de bord administrateur - Vue d'ensemble

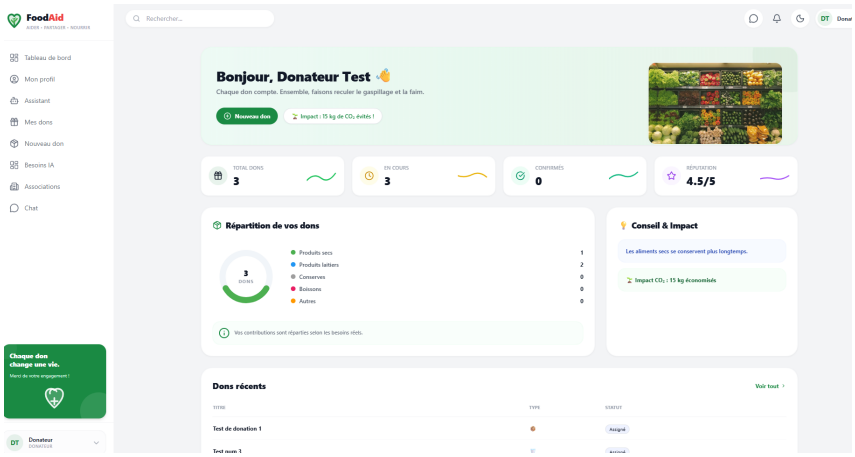


Figure 3.4 – Tableau de bord donateur - Gestion des dons

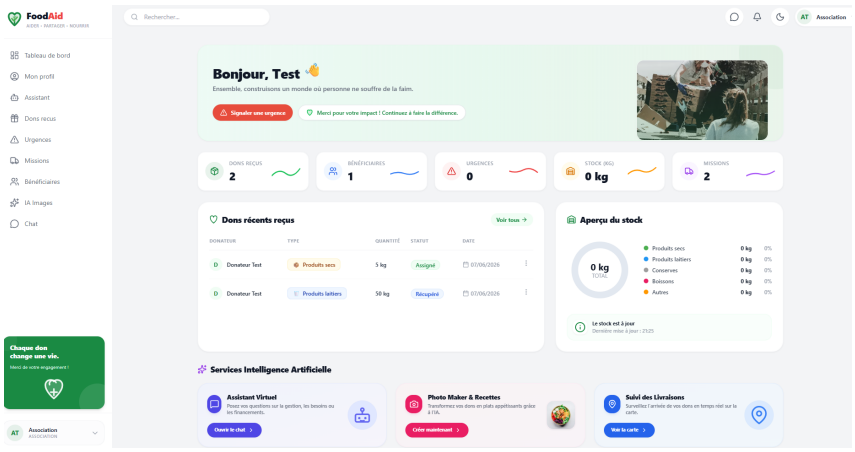


Figure 3.5 – Tableau de bord association - Suivi des aides

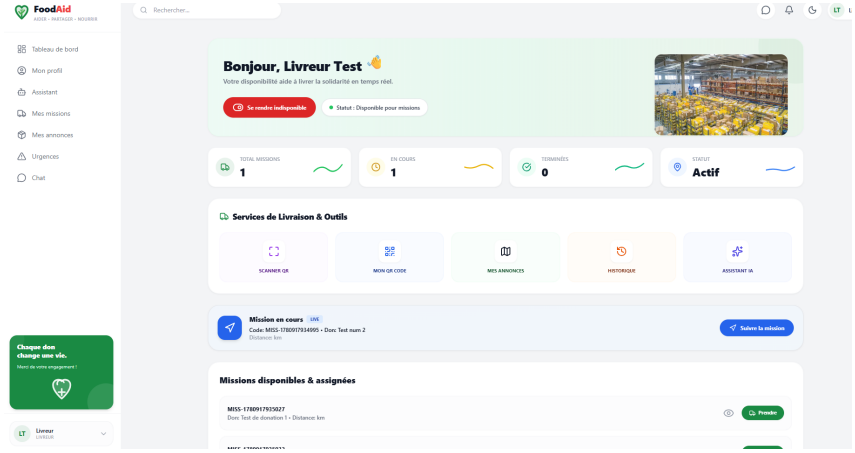


Figure 3.6 – Tableau de bord livreur - Missions de livraison

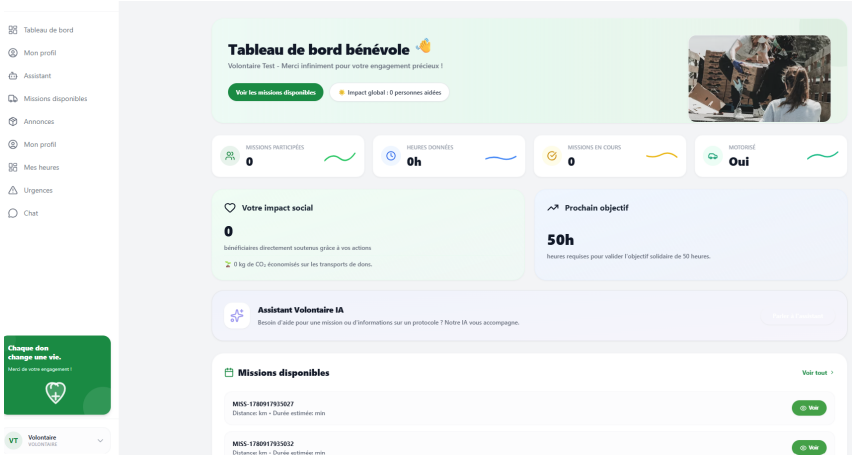


Figure 3.7 – Tableau de bord volontaire - Actions terrain

3.5 Tests fonctionnels

3.5.1 Ordre des tests

Les tests fonctionnels doivent suivre l'ordre suivant :

1. Démarrer PostgreSQL, le backend et le frontend ;
2. Vérifier la connexion avec les comptes de test ;
3. Vérifier les redirections par rôle ;
4. Créer un don en tant que donateur ;
5. Valider le don en tant qu'administrateur ;
6. Assigner le don à une association ;
7. Vérifier la création de la mission PENDING ;
8. Connecter un livreur vérifié et disponible ;
9. Accepter la mission ;
10. Démarrer la mission ;
11. Terminer la mission ;
12. Vérifier le QR Code ;
13. Scanner le QR Code ;
14. Confirmer la livraison ;
15. Tester les urgences ;
16. Tester les annonces et groupes ;
17. Tester le chat et le support ;
18. Tester les pages IA.

3.5.2 Tests de gestion des dons

Vérifier :

— Création d'un don ;
— Contrôle de la date de péremption ;
— Validation ;
— Rejet ;
— Assignation à une association ;
— Changement de statut ;
— Création de mission après assignation ;
— Annulation selon statut.

3.5.3 Tests de livraison et validation QR Code

Vérifier :

- Récupération des missions en attente ;
- Acceptation par un livreur disponible et vérifié ;
- Blocage si le livreur n'est pas disponible ou pas vérifié ;
- Démarrage de mission ;
- Passage du don à PICKED_UP ;
- Clôture de mission ;
- Passage du don à DELIVERED ;
- Génération du QR Code ;
- Scan unique ;
- Blocage du scan si déjà scanné ou expiré.

Conclusion du chapitre

Ce chapitre a présenté l'environnement de développement, l'architecture technique, la structure du projet, les modules implémentés, les interfaces utilisateur et les tests fonctionnels. La plateforme FoodAid est désormais opérationnelle avec l'ensemble des fonctionnalités prévues : authentification, gestion des dons, missions, QR Code, chat, IA et tableaux de bord.

Conclusion générale et perspectives

Bilan du projet

Le projet FoodAid a permis de développer une plateforme web complète de gestion et distribution des dons alimentaires, répondant aux besoins identifiés. Les objectifs fixés ont été atteints : création d'une plateforme fonctionnelle et sécurisée, gestion complète du cycle de vie des dons, coordination efficace entre les six acteurs (administrateur, donateur, association, livreur, volontaire, bénéficiaire), intégration de fonctionnalités avancées (IA, géolocalisation, QR Code, messagerie temps réel) et interfaces adaptées à chaque rôle.

Difficultés rencontrées

Les principales difficultés ont concerné l'assurance de la cohérence entre le frontend et le backend, l'alignement des noms de champs et des endpoints, la gestion des statuts des dons et des missions sans transition invalide, le rendu des missions visibles après assignation d'un don, ainsi que l'intégration des services IA avec des temps de réponse acceptables.

Apports du projet

Ce projet a permis de mettre en pratique l'ensemble des compétences acquises : conception d'une architecture full-stack (Spring Boot, React, PostgreSQL), mise en place de la sécurité JWT, développement d'API REST, gestion de rôles complexes, intégration de cartes GPS, génération et validation de QR Code, communication en temps réel via WebSocket, intégration de services IA et RAG, et méthodologie de test et validation.

Perspectives

Plusieurs axes d'amélioration sont envisagés pour enrichir la plateforme : développement d'une application mobile, optimisation avancée des itinéraires de livraison, ajout de notifications push, enrichissement du module bénéficiaire, et extension des fonctionnalités IA.

Références bibliographiques

1. Documentation officielle Spring Boot - https://spring.io/projects/spring-boot
2. Documentation officielle Spring Security - https://spring.io/projects/spring-security
3. Documentation officielle React - https://react.dev/
4. Documentation officielle Vite - https://vitejs.dev/
5. Documentation officielle PostgreSQL - https://www.postgresql.org/docs/
6. Documentation officielle Flyway - https://flywaydb.org/documentation/
7. Documentation officielle JWT - https://jwt.io/introduction
8. Documentation officielle Leaflet - https://leafletjs.com/
9. Documentation officielle Groq API - https://ai.google.dev/Groq-api/docs
10. Documentation officielle Tailwind CSS - https://tailwindcss.com/docs
11. Code source du projet FoodAid : backend Spring Boot, frontend React, migrations SQL et fichiers de configuration.